

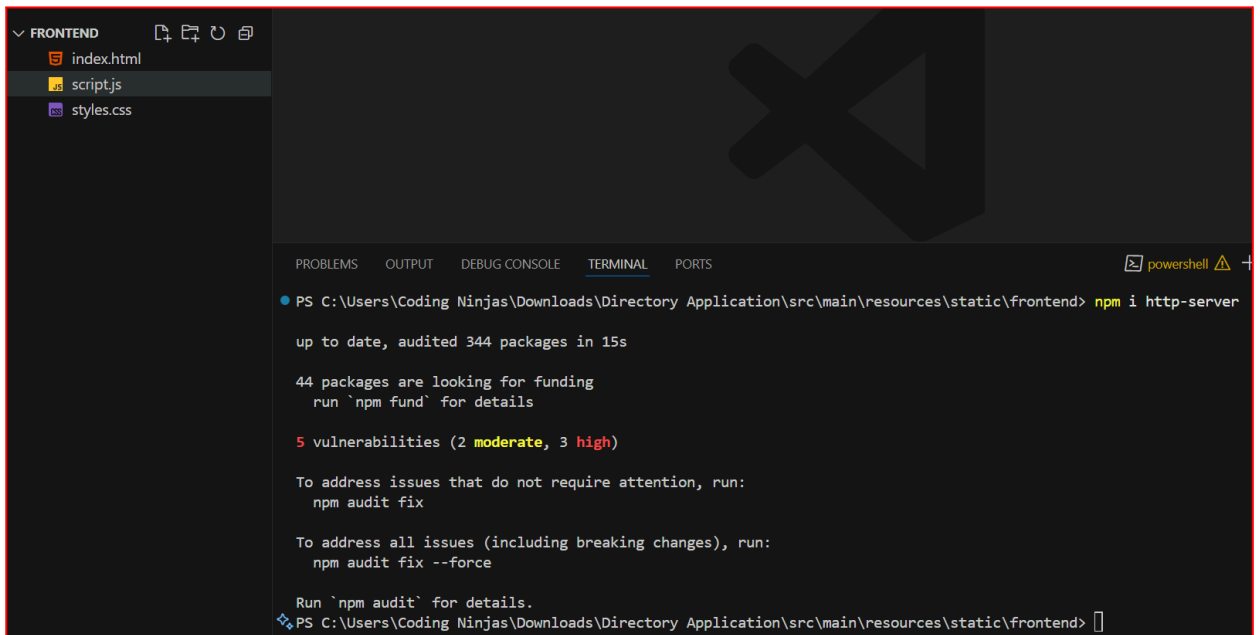
Integrating Frontend with Spring Boot

Installing http-server

- To run the front-end code, we first need to install an **HTTP server**. You can do this by running the following command in your project workspace:

[Directory App FrontEnd Code](#)

```
npm i http-server
```



The screenshot shows a Visual Studio Code editor with a file explorer on the left displaying 'index.html', 'script.js', and 'styles.css'. The main editor area shows a terminal window with the command 'npm i http-server' executed. The output indicates that 344 packages were audited in 15 seconds, 44 packages are looking for funding, and 5 vulnerabilities were found (2 moderate, 3 high). It also provides instructions on how to address these vulnerabilities using 'npm audit fix' or 'npm audit fix --force'.

Note: To run this command, you need to have Node.js installed on your system..

Setting up Node

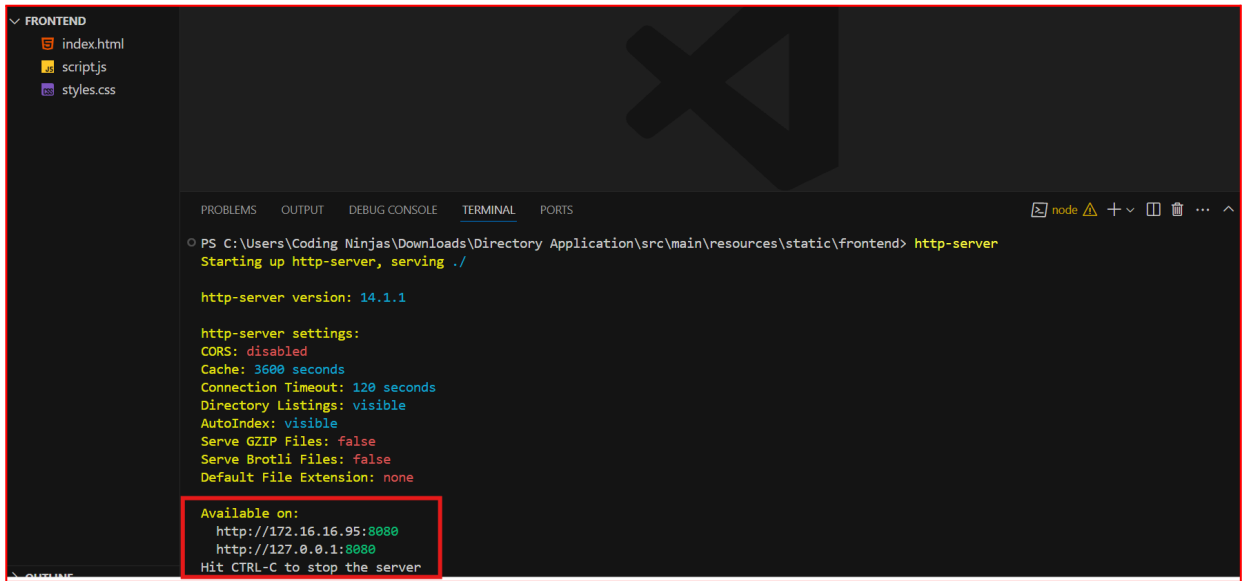
Steps to Install Node.js

1. Go to nodejs.org, the official Node.js website. [Link](#)
2. Choose between the LTS (Long Term Support) version of Node.js.
3. It is recommended to select the LTS version for better stability, especially for beginners.
4. Download the LTS installer that matches your operating system.
5. Follow the installation steps.
6. After the installation process, Node.js will be installed and ready to use.

7. To check the Node.js installation, open the terminal/command prompt.
8. Type `node -v` and press enter.
9. The installed version will be displayed.

Starting the http-server

- To start the server, you need to use the command **http-server**. Once the server is running, it will provide you with the available server links as shown below:



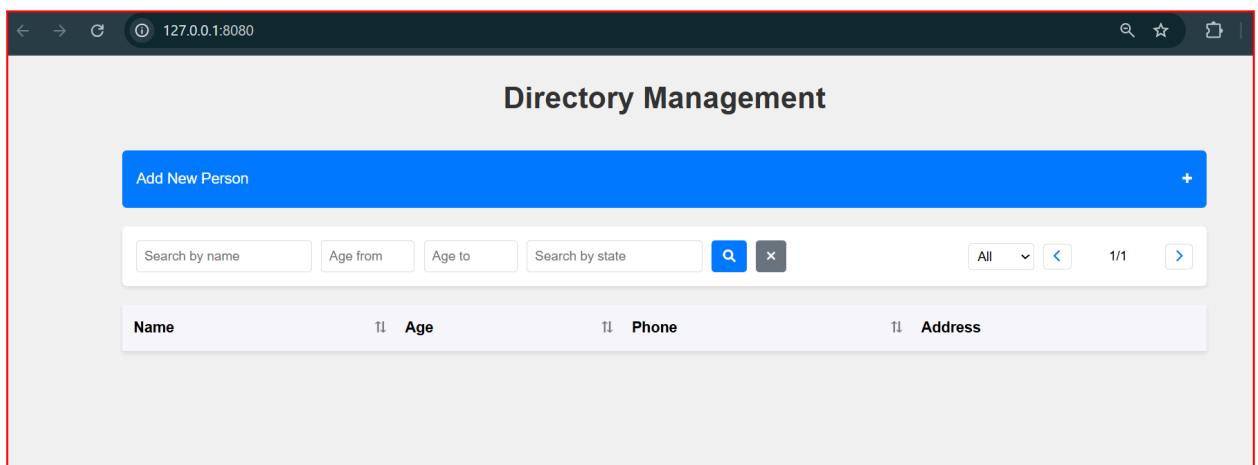
```
PS C:\Users\Coding Ninjas\Downloads\Directory Application\src\main\resources\static\frontend> http-server
Starting up http-server, serving ./

http-server version: 14.1.1

http-server settings:
CORS: disabled
Cache: 3600 seconds
Connection Timeout: 120 seconds
Directory Listings: visible
AutoIndex: visible
Serve GZIP Files: false
Serve Brotli Files: false
Default File Extension: none

Available on:
http://172.16.16.95:8080
http://127.0.0.1:8080
Hit CTRL-C to stop the server
```

- Visit either of the two links and verify if the directory management application is running or not. If you can see the same output, then your front-end setup is completed.



Web Config

This Java code configures your Spring Boot backend to accept requests from your http-server frontend. It does this by setting up **CORS (Cross-Origin Resource Sharing)** rules.

In web security, browsers enforce a "Same-Origin Policy." This policy blocks a frontend running on one origin (e.g., **`http://127.0.0.1:8080`** where your **`http-server`** is running) from making API calls to a backend running on a different origin (e.g., **`http://localhost:9090`** where your Spring Boot app might be running).

This **WebConfig** class tells your backend to relax this rule for your specific frontend.

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
            .allowedOrigins("http://127.0.0.1:8080")
            .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
            .allowedHeaders("*")
            .allowCredentials(true);
    }
}
```

Code Breakdown

Here's what each part of the code does:

- **@Configuration:** This annotation tells Spring that this class contains configuration settings for the application.
- **implements WebMvcConfigurer:** By implementing this interface, you can customize the Spring MVC framework, which handles web requests.
- **addCorsMappings(CorsRegistry registry):** This is the specific method you override to define your CORS rules.
- **registry.addMapping("/**"):** This rule applies to all endpoints in your application. The `/**` is a wildcard that matches every URL path.
- **.allowedOrigins("http://127.0.0.1:8080"):** This is the most important line for you. It explicitly gives permission to frontend code loaded from `http://127.0.0.1:8080` (the

default address for http-server) to make requests to your backend.

- **.allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS"):** This specifies which HTTP methods (like fetching data with GET or submitting a form with POST) are allowed from your frontend.
- **.allowedHeaders("*"):** This allows your frontend to send any HTTP headers (like Authorization for tokens or Content-Type) in its requests.
- **.allowCredentials(true):** This setting permits the browser to send credentials like cookies or HTTP authentication information along with the cross-origin request. This is necessary if your application uses sessions or cookie-based authentication.

In short, this configuration creates a bridge, allowing your **http-server** frontend and Spring Boot backend to communicate securely despite being on different origins.